
Seek Wander

SRE / DevOps / Cloud

Technical Briefing

Meeting Preparation Guide

Confidential — Internal Use Only

Document Type	Technical Architecture Briefing
Classification	Confidential
Audience	SRE / DevOps / Cloud Specialists
Stack	Next.js 14 · Vercel · Supabase · Anthropic · Clerk

Table of Contents

1. Infrastructure & Hosting
2. Deployment Pipeline
3. Database Architecture
4. Observability & Monitoring
5. External Dependencies & SLAs
6. Cost Architecture
7. Caching Strategy
8. Security Posture
9. Scalability Characteristics
10. Questions They Will Likely Ask You
11. Cheat Sheet — Quick Reference

1. Infrastructure & Hosting

What you're running on: Vercel (Serverless)

Your entire app runs on Vercel's serverless platform, not a traditional server. This is the most important concept to understand going into the meeting.

What 'serverless' actually means for your app:

- There is no persistent server. Each API request spins up a fresh function, executes, and dies
- You don't manage VMs, OS patches, load balancers, or capacity planning — Vercel handles all of it
- Your `/api/itinerary` route is a Node.js serverless function that Vercel runs on AWS Lambda under the hood
- Cold start problem: The first request after inactivity takes ~300-500ms longer because the runtime needs to warm up. Acceptable since AI generation takes 90+ seconds anyway

Your function timeout:

```
export const maxDuration = 290 // seconds
```

You explicitly set this to 290 seconds because the AI generation pipeline (Claude API call + Google Places enrichment) can take 90-150 seconds. Vercel Pro's ceiling is 300 seconds — 10 seconds of buffer.

Edge Network:

- Static assets (JS, CSS, images) are served from Vercel's CDN (200+ global edge locations)
- API routes run in a specific region (likely `iad1` — US East, Vercel default)
- Node.js runtime used (not edge runtime) because your routes use Prisma and crypto APIs not available on the edge

2. Deployment Pipeline

Your current setup:

```
git push -> GitHub -> Vercel auto-detects -> build -> deploy
```

Key facts:

- Zero-downtime deploys: Vercel builds the new version in parallel, then atomically switches traffic. Users mid-request on the old version finish normally
- Preview deployments: Every PR/branch gets its own unique URL — instant staging environments, no config required
- Rollback: One click in Vercel dashboard to redeploy any previous successful build
- Build command: `npm run build` -> runs next build + prisma generate (via postinstall in package.json)

The prisma generate postinstall — IMPORTANT:

```
"postinstall": "prisma generate"
```

This is critical. Prisma generates platform-specific binary clients. Without this, the Linux binary for Vercel's AWS Lambda environment would be missing and every DB query would crash. This runs automatically on every Vercel deploy.

Known gaps (they'll likely ask):

- No automated test suite running in CI
- No staging environment with production-like data
- No blue/green deployment strategy (Vercel handles this implicitly)

3. Database Architecture

Stack: Supabase (managed PostgreSQL) + Prisma ORM

Two connection URLs — this will definitely come up:

```
DATABASE_URL = postgresql://...@db.xxx.supabase.co:6543/postgres # PgBouncer (pooled)
DIRECT_URL    = postgresql://...@db.xxx.supabase.co:5432/postgres # Direct connection
```

Why two connections?

PostgreSQL has a hard limit on simultaneous connections. Serverless is dangerous for databases:

```
Traditional server: 1 server x 1 connection pool = controlled
Serverless:        1000 concurrent functions x 1 connection each = 1000 connections = DB crashes
```

PgBouncer (port 6543) is a connection pooler that sits between your functions and PostgreSQL. It maintains a small pool (e.g. 25 connections) and queues/reuses them. 1000 serverless functions -> PgBouncer -> 25 actual PostgreSQL connections.

Direct connection (port 5432) is used only by Prisma's db push / schema migrations during development. PgBouncer's transaction-mode pooling doesn't support the session-level protocol Prisma uses for migrations.

Prisma singleton pattern:

```
const globalForPrisma = global as unknown as { prisma: PrismaClient };
export const prisma = globalForPrisma.prisma ?? new PrismaClient();
if (process.env.NODE_ENV !== 'production') globalForPrisma.prisma = prisma;
```

Without this, Next.js hot-reload in development would create hundreds of PrismaClient instances (each opening its own connection pool), exhausting Supabase's connection limit within minutes.

Your three tables:

- **Trip** — user itineraries (immutable after save)
- **PlaceCache** — Google Places API responses (30-day TTL, reduces API spend)
- **CostLog** — every generation logged with token counts and dollar costs

Schema deployment: `npx prisma db push` — pushes directly to Supabase. **Note:** no migration files currently (they'll flag this as a gap for production).

4. Observability & Monitoring

What you have:

- console.info/warn/error logs visible in Vercel's function log viewer (real-time and historical)
- Structured log format: [itinerary] Claude generation: 94ms | in=2847 out=6234 tokens
- /admin/metrics — internal dashboard showing CostLog data (generations, spend, cache hit rate)
- Vercel's built-in analytics (Web Vitals, request volume, error rates)
- Log retention: 7 days on Vercel Pro

What the SRE will immediately flag as missing:

- No error tracking (Sentry, Datadog, Rollbar) — uncaught exceptions in production are invisible
- No uptime monitoring (UptimeRobot, Pingdom) — you don't know if your app goes down before users tell you
- No alerting — no PagerDuty/email alerts on error rate spikes or high latency
- No distributed tracing — can't trace a single request across the full pipeline
- No p50/p95/p99 latency tracking — no percentile data on generation times
- No long-term log storage/aggregation (Elastic, Loki, Datadog Logs)

5. External Dependencies & SLAs

Your app has 5 hard external dependencies. This is the most important reliability topic.

Dependency	Usage	SLA	Failure Impact
Anthropic API	Every generation	99.9%	Complete generation failure
Google Maps/Places	Every generation + map UI	99.9%	Enrichment fails, map blank
Supabase	Every request (auth, cache, save)	99.5%	Trips cannot save, cache misses
Upstash Redis	Every API request (rate limit)	99.9%	Rate limiting fails open or blocks all
Clerk	Every authenticated action	99.99%	Auth fails, users cannot access trips

Compounded availability — the key insight:

$$0.999 \times 0.999 \times 0.995 \times 0.999 \times 0.9999 = \sim 0.992 \quad (99.2\%)$$

Your theoretical availability ceiling is **~99.2%** even if your own code is perfect. This is called **dependency risk**. The SRE will raise this.

Your current failure handling:

- Anthropic timeout -> JSON self-heal attempt -> structured error response
- Google Places failure -> graceful null return, no photo/rating shown
- Redis failure -> currently would block all requests
- Supabase failure -> unhandled, would throw 500

6. Cost Architecture

Every generation costs real money, tracked in the CostLog table:

```
Claude input:   $3.00 / 1M tokens
Claude output: $15.00 / 1M tokens
Google Places Text Search: $0.032 / call
Google Places Details:      $0.017 / call
```

Typical generation cost:

- ~3,000 input tokens + ~6,000 output tokens = ~\$0.10 Claude cost
- ~20 Places calls (mix of cache hits/misses) = \$0.03-\$0.30 Google cost
- **Total per generation: ~\$0.10-\$0.40**

Cost protection layers:

- **Rate limiting** (5/user/hour) — prevents a single user from burning \$50 in an hour
- **PlaceCache** — Google API calls cached for 30 days. Cache hit rate visible in /admin/metrics
- **max_tokens: 8192** — hard cap prevents runaway long generations
- **CostLog** — every generation AWAITED before response (not fire-and-forget). Vercel freezes functions the instant the HTTP response returns

The **Zero-Latency Principle** is also a cost decision: one AI call per itinerary, never two. Live activity swapping would double per-session cost and was explicitly rejected at the architecture stage.

7. Caching Strategy

LAYER 1 — Google Places Cache (Supabase PlaceCache table)

- 30-day TTL on enrichment data (photos, ratings, opening hours)
- Reduces Google API cost from ~\$0.30/generation to near-zero for popular destinations
- Nightly Vercel Cron job (0 0 * * *) deletes rows older than 14 days
- Indexed on updatedAt — the nightly cleanup is $O(\log n)$ range scan, not $O(n)$ full table scan
- Cache key: "{normalized-place-name}{{normalized-city}"
- Stores raw photo_reference tokens (not full URLs) — survives API key rotation

LAYER 2 — sessionStorage Itinerary Cache (browser)

- After generation, full itinerary JSON saved to sessionStorage with a fingerprint
- Navigating back to /itinerary restores instantly — zero API call, zero cost
- Fingerprint includes all form fields — changing any single field invalidates the cache
- Session-scoped — clears automatically when browser tab closes

LAYER 3 — Next.js Static Asset CDN (Vercel Edge)

- All JS, CSS, fonts cached at the edge globally (200+ locations)
- next/image serves optimised WebP with cache headers
- getDestinationPhotoUrl() has next: { revalidate: 86400 } — 24-hour cache at Next.js Data Cache layer

8. Security Posture

9 implemented security layers:

1. Input Validation	Zod safeParse on all API POST bodies. Validates types, ranges, and enum values before any AI or DB call. safeParse (not parse) — never throws, never leaks stack traces.
2. Rate Limiting	Upstash Redis slidingWindow(5, '1 h') per userId. Sliding window prevents burst exploitation. Falls back to IP for anonymous users. Hard rejects if neither userId nor IP is available.
3. Prompt Injection Defence	User-supplied fields land in the user message role; system prompt is the authority. Blocklist regex blocks shell metacharacters while preserving international address characters.
4. IDOR Prevention	Every DB query filters by both resource ID and authenticated userId. Missing AND wrong-owner records return identical neutral 404. Atomic deleteMany({ id, userId }) eliminates TOCTOU race window.
5. API Key Split	Browser key (NEXT_PUBLIC_) restricted by HTTP referrer to your domains only. Server key (MAPS_SERVER_KEY) never sent to client.
6. HTTP Security Headers	X-Frame-Options: DENY (clickjacking), X-Content-Type-Options: nosniff (MIME sniffing), Referrer-Policy: strict-origin-when-cross-origin, Permissions-Policy: camera=(), microphone=(), geolocation=()
7. Cron Authentication	Bearer token (CRON_SECRET, 256-bit entropy) checked before any DB work.
8. Admin Route Protection	notFound() not redirect() — route existence not confirmed to non-admins.
9. Clerk Auth	All routes public in middleware; individual routes enforce auth independently. await auth() required (Promise in Clerk v6.39+).

Known gaps:

- No WAF (Web Application Firewall)
- No CSP — would break Google Maps JS SDK and Clerk
- 4 npm audit CVEs tracked — blocked on Clerk v7 + Next.js 15 coordinated upgrade

9. Scalability Characteristics

What scales automatically:

- Serverless functions scale to thousands of concurrent requests with zero config
- Vercel CDN scales infinitely for static assets
- Supabase scales with plan tier

What does NOT scale automatically:

- Anthropic API rate limits — per-key token-per-minute limits. Under high load would need a job queue (Redis Bull, Inngest) rather than direct API calls
- Supabase connection limit — PgBouncer helps but has its own ceiling. Under very high load would need dedicated connection pooler
- Redis rate limiter — single Upstash region; under extreme load could become a bottleneck

Honest scale answer: Current architecture handles hundreds of daily users comfortably. At thousands of concurrent AI generations, you'd need: a job queue system, response streaming, and possibly a dedicated DB proxy. That's a planned future architectural change.

10. Questions They Will Likely Ask You

Q: What's your RTO and RPO?

A: RTO (Recovery Time Objective) = ~5 minutes (Vercel redeploy from dashboard). RPO (Recovery Point Objective) = near-zero for user trips (Supabase has point-in-time recovery). PlaceCache data could be lost and would self-rebuild from Google over 2-4 weeks of normal usage.

Q: How do you deploy schema changes?

A: Currently using npx prisma db push — pushes schema changes directly to Supabase. Acknowledged gap: no migration files, no rollback capability. For production scale, would migrate to prisma migrate with explicit versioned migration files.

Q: What happens if Anthropic goes down?

A: User sees a structured error message ('Something went wrong'). No fallback provider, no queuing for retry, no status page integration. Acceptable for V1 — would need addressing at scale (fallback to GPT-4o, async job queue).

Q: How do you handle secrets rotation?

A: Vercel env vars updated in dashboard, redeploy required. Google API key rotation hardened — PlaceCache stores raw photo_reference tokens instead of full URLs with embedded keys, so photo cache survives key rotation without manual data migration.

Q: Where are your logs?

A: Vercel function logs, 7-day retention, structured format with [itinerary] prefix for filtering. Gap: no centralised log aggregation, no long-term retention. Would add Sentry for error tracking and Datadog or Axiom for log aggregation as next step.

Q: How do you handle database migrations in production?

A: Currently prisma db push. They will recommend moving to prisma migrate dev + prisma migrate deploy in CI/CD pipeline for versioned, reversible schema changes.

Q: What's your on-call process?

A: Currently none — single founder, no formal incident response process. Monitoring is manual (check Vercel dashboard). Gap acknowledged — would implement PagerDuty or similar at scale.

11. Cheat Sheet — Quick Reference

Topic	Your Answer
Hosting	Vercel serverless, built on AWS Lambda
Database	Supabase PostgreSQL, PgBouncer pooling, Prisma ORM
Frontend	Next.js 14 App Router, deployed to Vercel CDN
Auth	Clerk v6, modal-only, all routes public in middleware
AI Model	Anthropic Claude (claude-sonnet-4-6), single upfront call, no streaming
Cost per generation	\$0.10-\$0.40, tracked in CostLog table
Caching	3 layers: DB (Google Places 30d), browser (sessionStorage), CDN (assets)
Deployments	Git push -> Vercel auto-build, zero-downtime, instant rollback
Monitoring	Vercel logs + /admin/metrics dashboard. No Sentry yet
Biggest gap	No error tracking, no uptime monitoring, no alerting
Scale ceiling	Comfortable to ~500 DAU. AI job queue needed beyond that
Function timeout	290 seconds (Vercel Pro max is 300s)
DB connections	PgBouncer pool (25 connections) handles unlimited serverless concurrency
Security layers	9 layers: Zod, rate limiting, prompt injection, IDOR, key split, HTTP headers, cron auth, admin protection, Clerk
Theoretical uptime	~99.2% (compounded dependency SLAs)

Key Terms to Use Confidently:

Serverless / Cold start / Function timeout	PgBouncer / Connection pooling / Transaction mode
IDOR (Insecure Direct Object Reference)	RT0 / RPO (Recovery Time / Point Objectives)
p50 / p95 / p99 latency percentiles	Sliding window rate limiting
Zero-downtime deployment / Atomic traffic switch	Dependency risk / Compounded availability
Distributed tracing	prisma migrate vs prisma db push